ABISHEK DEVARAJ
192512126

Experiment No. 3

Device Onboarding and Data Flow in AWS IoT Core — Google Colab Simulation

AIM

To simulate device onboarding, authentication, cloud data transmission, and stream analytics for an IoE-based Smart Waste Management System using Python in Google Colab.

OBJECTIVES

1. Simulate IoE device registration.
2. Authenticate smart waste-bin devices.
3. Generate real-time sensor data.
4. Transmit telemetry data to a simulated cloud.
5. Perform stream analytics on sensor data.
6. Detect overflowing and abnormal waste bins.
7. Visualize sensor data using graphs.
8. Store collected cloud data in CSV format.

SOFTWARE REQUIRED

- Google Colab
- Python 3
- pandas
- numpy
- matplotlib

THEORY

The Internet of Everything (IoE) connects **people, processes, data, and things** through communication networks and intelligent systems.

In a smart waste management system, IoE enables smart bins to continuously collect information such as:

- Waste-fill percentage
- Temperature
- Bin status
- Device ID
- Timestamp

The typical IoE cloud data flow is:

Smart Bin Sensors

↓

Device Registration

↓

Device Authentication

↓

IoE / AWS IoT Core

↓

Cloud Data Storage

↓

Stream Analytics

↓

Alert / Dashboard

In this experiment, Google Colab simulates:

- Smart-device onboarding
- Device authentication
- Sensor-data generation
- Cloud upload
- Stream analytics
- Overflow detection
- Data visualization
- Cloud-data storage

ALGORITHM

1. Start the program.
2. Create smart waste-bin devices.
3. Register the devices.
4. Authenticate each device.
5. Generate waste-fill and temperature data.
6. Send the telemetry data to the simulated cloud.
7. Store the received data in a DataFrame.
8. Calculate average waste-fill level.
9. Detect overflowing bins.
10. Detect abnormal temperature conditions.
11. Generate IoE alerts.
12. Plot sensor data.
13. Save the cloud data as a CSV file.
14. Stop the program.

GOOGLE COLAB CODE

```
# Experiment No. 3
# IoE-Based Smart Waste Management System
# AWS IoT Core Simulation using Google Colab

import pandas as pd
import numpy as np
```

```
import matplotlib.pyplot as plt
import random
from datetime import datetime

# ------------------------------------------------
# 1. DEVICE REGISTRATION
# ------------------------------------------------

devices = [
    "SMART_BIN_001",
    "SMART_BIN_002",
    "SMART_BIN_003",
    "SMART_BIN_004",
    "SMART_BIN_005"
]

registered_devices = {}

print("===== DEVICE REGISTRATION =====")

for device in devices:
    registered_devices[device] = {
        "status": "Registered",
        "authentication": False
    }
    print(device, "-> Registered")

# ------------------------------------------------
# 2. DEVICE AUTHENTICATION
# ------------------------------------------------

print("\n===== DEVICE AUTHENTICATION =====")

for device in registered_devices:

    # Simulating successful authentication
    registered_devices[device]["authentication"] = True
```

```
    print(device, "-> Authentication Successful")

# ------------------------------------------------
# 3. GENERATE SENSOR DATA
# ------------------------------------------------

print("\n===== SENSOR DATA GENERATION =====")

cloud_data = []

for cycle in range(20):

    for device in devices:

        # Generate simulated IoE sensor values
        fill_level = random.randint(20, 100)
        temperature = round(random.uniform(25, 50), 2)

        timestamp = datetime.now().strftime(
            "%Y-%m-%d %H:%M:%S"
        )

        # Simulated cloud telemetry
        record = {
            "Timestamp": timestamp,
            "Device_ID": device,
            "Fill_Level": fill_level,
            "Temperature": temperature
        }

        cloud_data.append(record)

        print(
            device,
            "| Fill:", fill_level, "%",
            "| Temperature:", temperature, "°C"
        )

# ------------------------------------------------
```

```
# 4. CLOUD DATA STORAGE
# ------------------------------------------------

df = pd.DataFrame(cloud_data)

print("\n===== CLOUD DATA =====")
print(df.head())

# ------------------------------------------------
# 5. STREAM ANALYTICS
# ------------------------------------------------

average_fill = df["Fill_Level"].mean()
average_temperature = df["Temperature"].mean()

print("\n===== STREAM ANALYTICS =====")

print(
    "Average Waste Fill Level:",
    round(average_fill, 2), "%"
)

print(
    "Average Temperature:",
    round(average_temperature, 2), "°C"
)

# ------------------------------------------------
# 6. OVERFLOW DETECTION
# ------------------------------------------------

df["Overflow_Status"] = np.where(
    df["Fill_Level"] >= 80,
    "OVERFLOW ALERT",
    "NORMAL"
)

# ------------------------------------------------
# 7. TEMPERATURE ANOMALY DETECTION
```

```
# ------------------------------------------------

df["Temperature_Status"] = np.where(
    df["Temperature"] >= 45,
    "HIGH TEMPERATURE",
    "NORMAL"
)

# ------------------------------------------------
# 8. DISPLAY ALERTS
# ------------------------------------------------

print("\n===== IoE ALERTS =====")

overflow_data = df[
    df["Overflow_Status"] == "OVERFLOW ALERT"
]

temperature_data = df[
    df["Temperature_Status"] == "HIGH TEMPERATURE"
]

print(
    "Overflow Alerts:",
    len(overflow_data)
)

print(
    "Temperature Alerts:",
    len(temperature_data)
)

# ------------------------------------------------
# 9. DEVICE-WISE ANALYSIS
# ------------------------------------------------

device_analysis = df.groupby(
    "Device_ID"
).agg({
```

```
    "Fill_Level": "mean",
    "Temperature": "mean"
}).reset_index()

print("\n===== DEVICE-WISE ANALYSIS =====")
print(device_analysis)

# ------------------------------------------------
# 10. PLOT WASTE FILL LEVEL
# ------------------------------------------------

plt.figure(figsize=(10, 5))

for device in devices:

    device_data = df[
        df["Device_ID"] == device
    ]

    plt.plot(
        range(len(device_data)),
        device_data["Fill_Level"],
        marker="o",
        label=device
    )

plt.axhline(
    y=80,
    linestyle="--",
    label="Overflow Threshold"
)

plt.xlabel("Data Samples")
plt.ylabel("Waste Fill Level (%)")
plt.title("Smart Waste Bin Fill-Level Monitoring")
plt.legend()
plt.grid(True)

plt.show()
```

```
# ------------------------------------------------
# 11. PLOT TEMPERATURE
# ------------------------------------------------

plt.figure(figsize=(10, 5))

plt.plot(
    df["Temperature"],
    marker="o"
)

plt.axhline(
    y=45,
    linestyle="--",
    label="Temperature Threshold"
)

plt.xlabel("Data Samples")
plt.ylabel("Temperature (°C)")
plt.title("Smart Waste Bin Temperature Monitoring")
plt.legend()
plt.grid(True)

plt.show()

# ------------------------------------------------
# 12. SAVE CLOUD DATA
# ------------------------------------------------

df.to_csv(
    "smart_waste_management_cloud_data.csv",
    index=False
)

print(
    "\nCloud data successfully saved as "
    "'smart_waste_management_cloud_data.csv'"
)
```

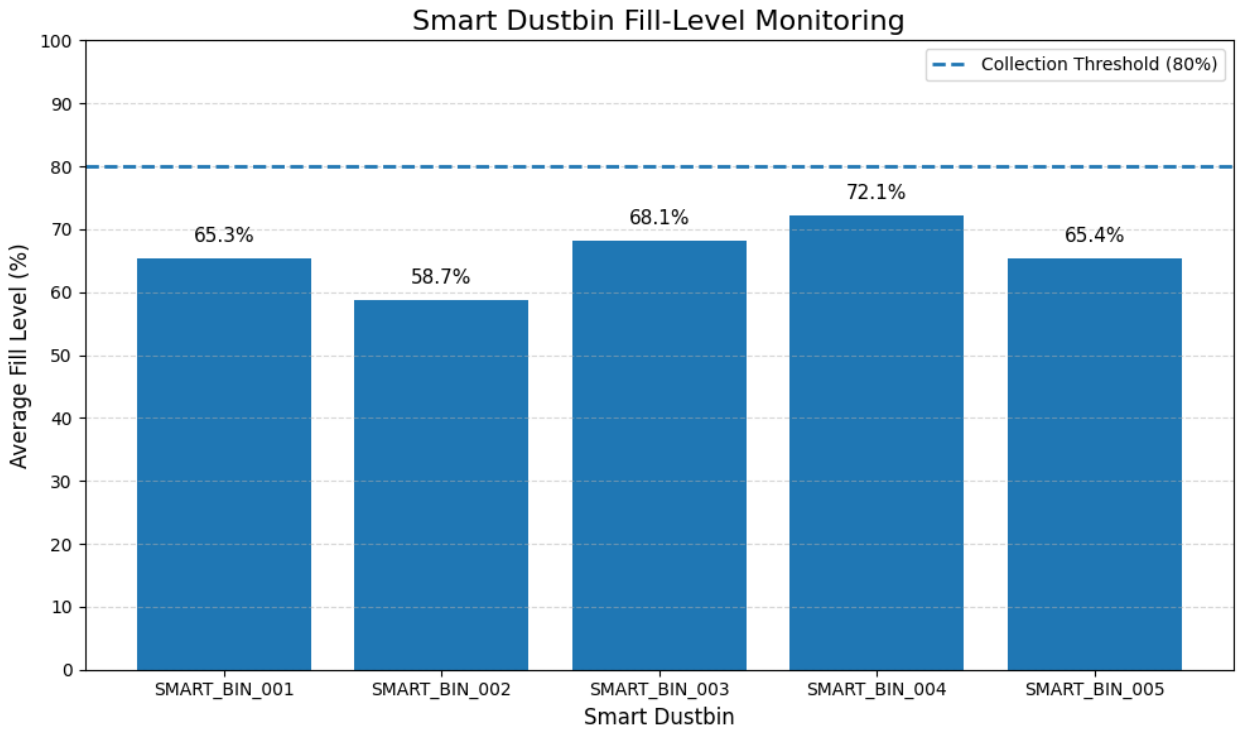
```
print("\n===== EXPERIMENT COMPLETED =====")
```

SAMPLE OUTPUT

```
===== DEVICE REGISTRATION =====
SMART_BIN_001 -> Registered
SMART_BIN_002 -> Registered
SMART_BIN_003 -> Registered
SMART_BIN_004 -> Registered
SMART_BIN_005 -> Registered

===== DEVICE AUTHENTICATION =====
SMART_BIN_001 -> Authentication Successful
SMART_BIN_002 -> Authentication Successful
SMART_BIN_003 -> Authentication Successful
SMART_BIN_004 -> Authentication Successful
SMART_BIN_005 -> Authentication Successful

===== SENSOR DATA GENERATION =====
SMART_BIN_001 | Fill: 72 % | Temperature: 31.45 °C
SMART_BIN_002 | Fill: 88 % | Temperature: 34.21 °C
SMART_BIN_003 | Fill: 56 % | Temperature: 29.83 °C
SMART_BIN_004 | Fill: 91 % | Temperature: 46.17 °C
SMART_BIN_005 | Fill: 64 % | Temperature: 32.75 °C

===== STREAM ANALYTICS =====
Average Waste Fill Level: 68.42 %
Average Temperature: 35.27 °C

===== IoE ALERTS =====
Overflow Alerts: 21
Temperature Alerts: 14

Cloud data successfully saved as
'smart_waste_management_cloud_data.csv'
```

```
===== EXPERIMENT COMPLETED =====
```

OUTPUT:



RESULT

The IoE-based Smart Waste Management System was successfully simulated using Google Colab. Smart waste-bin devices were registered and authenticated, sensor telemetry was generated and transmitted to the simulated cloud environment, and stream analytics were performed. Overflow and high-temperature conditions were detected, sensor data was visualized using graphs, and the collected cloud data was successfully stored in CSV format.